OBLIVION
Blossom

S.I.G.S.M.

Administración de Sistemas Operativos

Oblivion Blossom Studio

Rol	Apellido	Nombre	C.I	Email
Coordinador	Vigo Silveira	Avril Agustina	5.361.073-5	avrilvigo2003@gmail.com
Sub-Coordinador	Pereyra Goncalvez	Candela Casandra	5.697.768-3	candelapereyra2007@gmail.com
Integrante 1	Acosta Díaz	Lucía Yasmin	5.666.263-4	luciayasminacostadiaz@gmail.com
Integrante 2	Franco Barros	Diannis D'Andrea	6.574.324-9	diannisdandrea@gmail.com

Docente: Vazquez, Gabriel

Fecha de Culminación 02/09/2026

SEGUNDA ENTREGA

S.I.G.S.M. I.S.B.O. 3MK

Índice

Índice **1**
1. Introducción al Documento **3**
2. Objetivos del Proyecto **3**
2.1. Objetivo General 3
2.2. Objetivos Específicos 3
2.2.1. Gestión para Documentación de Pacientes: 3
2.2.2. Trazabilidad de Vehículos: 4
3. Sobre el Equipo **4**
3.1. Carta de Presentación 5
3.2. Roles y Funciones de los Integrantes 6
4. Hardware necesario para los servidores: **8**
5. Costos de las diferentes formas de obtener el servidor requerido: **9**
6. Descripción de Vulnerabilidades en Versiones Anteriores de Linux **11**
6.1. Introducción 11
6.2. Vulnerabilidades Históricas Relevantes 11
6.2.1 Dirty COW (CVE-2016-5195) 11
6.2.2 Dirty Pipe (CVE-2022-0847) 12
6.2.3 Spectre y Meltdown (CVE-2017-5753 / CVE-2017-5754) 12
6.2.4 CVE-2024-1086 (Escalada de privilegios en Netfilter) 13
6.2.5 CopyFail (CVE-2026-31431) 13
6.3. Conclusión 14
7. Configuración de Usuarios y Grupos **15**
7.1. Definición de Roles y Criterio de Permisos 15
7.2. Creación de Grupos Funcionales 15
7.3. Asignación de Permisos sobre Archivos y Directorios 16
7.4. Uso de Privilegios Elevados 16
8. Respaldo a Largo Plazo y Alta Disponibilidad **17**
8.1. Selección de medios de almacenamiento para respaldo a largo plazo 17
8.2. Estrategia de retención 18
8.3. Mecanismos de alta disponibilidad 18
8.4. Plan de recuperación ante desastres (RTO/RPO) 19
9. Mecanismos de Backup para los Datos de la Aplicación **20**
9.1. Descripción general del script 20
9.2. Backup de archivos propios de la aplicación (configuraciones, uploads, logs) 21
9.3. Cómo se almacenan los backups 21

S.I.G.S.M. **I.S.B.O.** **3MK**

9.4. Automatización mediante cron 22
9.5. Restauración de datos 22
Bibliografía **23**
Anexos **23**

1. Introducción al Documento

El presente documento detalla la planificación de infraestructura, dimensionamiento de hardware y políticas de seguridad del sistema operativo para el proyecto que llevan a cabo alumnos de bachillerato tecnológico BT para el Hospital de Clínicas en beneficio de egreso. Su propósito es definir un entorno de producción eficiente, económicamente viable y protegido para el correcto despliegue de la plataforma.

A lo largo de este informe se abordan el **Hardware necesario para los servidores**, justificando la elección de un servidor a partir de un análisis de la demanda diaria estimada de traslados, también ver **Costos y formas de obtención, s**e evalúan comparativamente las ventajas operativas y financieras de adquirir un servidor propio y la **Seguridad y vulnerabilidades en Linux,** realizando una auditoría de fallos críticos históricos y recientes del kernel fundamentando así la decisión de implementar la distribución elegida.

2. Objetivos del Proyecto

2.1. Objetivo General

Para este proyecto se debe diseñar, desarrollar e implementar dos módulos de software bajo entorno GNU/Linux para el cliente, en este caso el Hospital de Clínicas, integrándose a sus servidores actuales, en los cuales se piden solucionar la gestión de ambulancias y vehículos del hospital realizando un seguimiento en tiempo real, por un lado y por el otro digitalizar documentos de interés de los pacientes para reducir la cantidad de uso de papel en ambos casos, para ayudar al personal administrativo en la gestión y optimizarla.

2.2. Objetivos Específicos

2.2.1. Gestión para Documentación de Pacientes:

Generar códigos QR específicos para cada documento de índole informativo médico (indicaciones, preparaciones, procedimientos y encuestas) para el acceso del paciente mediante dispositivos móviles en lugar de usar un folleto y generar un mayor gasto en papel. También desarrollar un apartado de encuestas

anónimas para recopilar valoraciones de los usuarios, para medir el índice de satisfacción.

2.2.2. Trazabilidad de Vehículos:

Gestionar y seguir el traslado de pacientes, u otros elementos biológicos o químicos. Se deberá registrar el conductor, asistente enfermero, paciente o elemento, lugar de origen, hora de salida, destino, hora de llegada y se debe de mostrar el estado actual del vehículo, su disponibilidad y ruta. Depende de lo que se traslade es el vehículo que se vaya a usar, al igual que los insumos y equipamiento necesarios para cada situación, en el caso de llevar un paciente se deberá registrar mediante su CI desde una API externa.

3. Sobre el Equipo

Oblivion Blossom Studio está conformado por integrantes diversos y capacitados para cubrir las áreas necesarias en el desarrollo de este proyecto. La empresa pretende ser inclusiva y fiel a la esencia de las personas que la componen. A pesar de ser un equipo que tiene fecha de disolución, en este corto periodo de tiempo aspira a superar los desafíos que se le presente. Al igual que las flores florecen y se marchitan, siempre vuelven a florecer y aunque el equipo tome caminos separados brillarán donde sea que estén, esa es la verdadera esencia de Oblivion Blossom.

3.1. Carta de Presentación

Oblivion Blossom Studio

Montevideo, 11 de mayo de 2026

Docente: Brandon Cairús
Asignatura: Ingeniería de Software
Instituto Superior Brazo Oriental

A continuación, los alumnos de tercero 3MK del turno vespertino del Instituto Superior Brazo Oriental nos presentamos ante usted, con el fin de informar la creación del grupo Oblivion Blossom Studio. Los correspondientes integrantes con sus roles son los siguientes:

A continuación, se detalla dicha integración y roles del grupo:

ROL	C.I	APELLIDO	NOMBRE	E-MAIL
Coordinador	53610735	Vigo Silveira	Avril Agustina	avrilvigo2003@gmail.com
Subcoordinador	56977683	Pereyra Goncalvez	Candela Casandra	pereyracandela2007@gmail.com
Integrante 1	56662634	Acosta Diaz	Lucia Yasmin	luciayasminacostadiaz@gmail.com
Integrante 2	65743249	Franco Barros	Diannis D'Andrea	diannisdandrea@gmail.com

Por contacto al correo: oblivionblossomstudio@gmail.com
Firmas:

COORDINADOR — SUBCOORDINADOR

INTEGRANTE 1 — INTEGRANTE 2

Oblivion Blossom Studio ISBO 3MK



VIGO, AVRIL
COORDINADOR



PEREYRA, CANDELA
SUBCOORDINADOR



ACOSTA, LUCÍA
INTEGRANTE 1



FRANCO, DIANNIS
INTEGRANTE 2

Oblivion Blossom Studio ISBO 3MK

3.2. Roles y Funciones de los Integrantes



Avril Vigo — Coordinadora

Como coordinadora del equipo, Avril es la representante oficial del grupo ante docentes y la institución. Es la responsable de la toma de decisiones generales, la organización de las entregas y la comunicación con los distintos actores del proyecto. Además, gracias a su capacidad de redacción técnica avanzada y su buen análisis ortográfico, lidera la elaboración de la documentación técnica del proyecto, garantizando que esta sea clara, coherente y de calidad. Su buen ojo para la colorimetría y los elementos visuales también aporta al diseño y presentación de las interfaces del sistema, cuidando la estética y simetría de la página web.



Candela Pereira — Subcoordinador

Como subcoordinador, Candela reemplaza a la coordinadora ante cualquier circunstancia que lo requiera y colabora activamente en la toma de decisiones del equipo. Es el principal responsable del desarrollo técnico del sistema, dado que cuenta con sólidos conocimientos en lógica de programación y manejo de herramientas como Visual Studio Code y XAMPP. Su capacidad para desarrollar código de forma eficiente y su comprensión de los modelos entidad relación la posicionan como el líder técnico del proyecto en el área de programación y base de datos.



Lucía Acosta — Integrante 1

Lucía es la responsable del área investigativa del proyecto. Su capacidad avanzada de investigación y su criterio para analizar documentos le permiten extraer la información más relevante y útil para el desarrollo del proyecto. Esto es fundamental para la elaboración del marco teórico, el estudio de factibilidad, la especificación de requerimientos y cualquier otro apartado que requiera una base sólida de información bien fundamentada.



Diannis Franco — Integrante 2

Diannis aporta al equipo sus destacadas habilidades de liderazgo y gestión interna, siendo un pilar fundamental para mantener la organización y la buena comunicación dentro del grupo. Su perfil facilita la resolución de conflictos y contribuye a que el equipo trabaje de forma armoniosa y productiva. Además colabora en la coordinación de tareas y el seguimiento del cronograma, asegurando que los plazos se cumplan y que cada integrante sepa en todo momento qué debe hacer.

4. Hardware necesario para los servidores:

Un único servidor:

- CPU: 8 núcleos.
- RAM: 16 GB.
- SSD: 1 TB.
- Red Gigabit.

-Proponemos un servidor con procesador de 8 núcleos, 16 GB de memoria RAM y 1 TB de almacenamiento SSD, ya que podrá gestionar documentos por códigos QR, encuestas de satisfacción y registros de traslados de ambulancias, con un rendimiento adecuado para muchos usuarios.

INVESTIGACIÓN: *Estudios globales indican que en hospitales de alta complejidad, solo entre el 8% y el 10% de los pacientes de emergencia utilizan ambulancia [1,2]. El 90% restante de los pacientes asiste por sus propios medios, utilizando transporte público, taxis o vehículos particulares [2]. Puede encontrar más detalles en los estudios publicados en ScienceDirect y PubMed Central (PMC).*

La proyección de traslados aproximada:

Al aplicar ese factor del 8% al 10% sobre las 120-150 consultas diarias del Clínicas.

-En caso de baja demanda (120 × 0.08): 10 ambulancias de emergencia al día.

-Y si es de alta demanda (150 × 0.10): 15 ambulancias de emergencia al día.

Y a las emergencias se le suman entre 6 y 9 vehículos diarios por traslados planificados.

-Consideramos que podría procesar los tres módulos, el estimado diarios no pasa de 20 traslados y la página donde se encuentran los documentos digitales de tamaño promedio y las encuestas, porque consume pocos recursos informáticos.

Procesador de 8 núcleos:

-Este procesador permite que varias tareas se realicen al mismo tiempo, mientras dos núcleos se encargan del sistema operativo, el resto se encarga de procesar las respuestas de las encuestas y los archivos pdf.

Memoria RAM de 16GB:
-Esta memoria permite fluidez entre multitareas, da la capacidad para mantener el software de encuestas, y documentos simultáneamente, además de los navegadores y la carga de documentos en pdf que cargaran por completo en la memoria ram.

Almacenamiento *SSD (1T)+ 1 TB de respaldo):*
-se utilizará un SSD de 1 TB como almacenamiento principal del servidor y un segundo SSD de 1TB destinado a copias de seguridad, permite conservar copias anteriores de la información y recuperar archivos que hayan sido eliminados o modificados accidentalmente.

RED_Gigabit Ethernet :
-Esta red garantiza la máxima velocidad de internet,ahorra las interferencias del WiFi y ofrece estabilidad total, esto nos asegura una sincronización inmediata en la nube y evita la pérdida de datos al subir o bajar informes médicos o formularios.

5. Costos de las diferentes formas de obtener el servidor requerido:

Comprar un servidor propio (*bare metal****): USD 634 – USD 977***

- *CPU: 8 núcleos. _ USD 272 / USD 290*
- *RAM: 16 GB. _ USD 47 / USD 207*
- *SSD principal 1TB _ USD 150 / USD 225*
- *SSD backup 1TB _ USD 150 / USD 225*
- *Red Gigabit. _ USD 15 / USD 30*

Costos Adicionales: USD 60 – USD 150

- *Limpieza y revisión de ventiladores: USD 20–50 cada 6 meses*
- *Revisión de SSD, RAM y hardware: USD 20–50 anual*

Costos de Reemplazo: USD 20 – USD 275

- *Reemplazo de ventiladores: USD 20–50*
- *Reemplazo de SSD defectuoso: USD 150–225*

Servidor propio (*Máquina Virtual: Proxmox****): USD 634 - USD 977***

- *CPU: 8 núcleos. _ USD 272 / USD 290*
- *RAM: 16 GB. _ USD 47 / USD 207*
- *SSD principal 1TB _ USD 150 / USD 225*
- *SSD backup 1TB _ USD 150 / USD 225*
- *Red Gigabit. _ USD 15 / USD 30*
- *Proxmox VE: _ USD 0*

Costos Adicionales: USD 60 – USD 150

- *Limpieza y revisión de ventiladores: USD 20–50 cada 6 meses*
- *Revisión de SSD, RAM y hardware: USD 20–50 anual*

Costos de Reemplazo: USD 20 – USD 275

- *Reemplazo de ventiladores: USD 20–50*
- *Reemplazo de SSD defectuoso: USD 150–225*

Servidor contratado (*Bare Metal dedicado****): Annual USD 550***

- *CPU: Intel Xeon 3.0 GHz — incluido*
- *RAM: 16 GB DDR4 — incluido*
- *SSD principal: 1 TB — incluido*
- *SSD para backup: 1TB — incluido*
- *Red Gigabit: 1 Gbps — incluida*
- *Mantenimiento del hardware_ incluido*
- *Costo mensual: _ USD 46*
- *Costo anual: _ USD 550*

S.I.G.S.M. I.S.B.O. 3MK

Costos adicionales: USD 0

- *Reemplazos incluido por el proveedor.*
- *Mantenimiento físico del servidor incluido por el proveedor.*

6. Descripción de Vulnerabilidades en Versiones Anteriores de Linux

6.1. Introducción

A lo largo de su historia, el kernel de Linux ha presentado diversas vulnerabilidades de seguridad en sus versiones anteriores. Si bien Linux es considerado uno de los sistemas operativos más seguros del mercado, ningún software está exento de fallos. El conocimiento de estas vulnerabilidades es fundamental para comprender la importancia de mantener el sistema operativo actualizado y contar con soporte técnico oficial que garantice la aplicación oportuna de parches de seguridad.

A continuación se describen las vulnerabilidades más relevantes detectadas en versiones anteriores del kernel de Linux.

6.2. Vulnerabilidades Históricas Relevantes

6.2.1 Dirty COW (CVE-2016-5195)

Año de descubrimiento: 2016
Versiones afectadas: Kernel de Linux desde la versión 2.6.22 (lanzada en 2007) hasta las versiones previas al parche de octubre de 2016.
Nivel de gravedad: Crítico

Descripción: Dirty COW (Copy-On-Write) fue descubierta por el investigador Phil Oester y es considerada una de las vulnerabilidades de escalada de privilegios más graves detectadas en la historia de Linux. El fallo se encontraba en el subsistema de memoria del kernel, específicamente en el mecanismo de copia virtual de operaciones de escritura.

Esta vulnerabilidad permitía que un usuario sin privilegios pudiera escribir y acceder a partes del sistema reservadas para otros usuarios, incluyendo el

usuario root. En la práctica, archivos que eran de solo lectura podían ser modificados por usuarios no autorizados, lo que representaba un riesgo crítico de seguridad.
Cabe destacar que la vulnerabilidad existió en el kernel durante aproximadamente 9 años antes de ser descubierta públicamente. El primer parche lanzado resultó ser defectuoso e incompatible con otras funciones del kernel, lo que requirió una corrección adicional.

6.2.2 Dirty Pipe (CVE-2022-0847)

Año de descubrimiento: 2022
Versiones afectadas: Kernel de Linux desde la versión 5.8 en adelante, hasta la aplicación del parche.
Nivel de gravedad: Alto

Descripción: Dirty Pipe fue descubierta en 2022 y es considerada una de las vulnerabilidades más graves de los últimos años en Linux. Su nombre hace referencia tanto a Dirty COW, con la que guarda similitudes, como al mecanismo de pipes (tuberías) del sistema Linux utilizado durante su explotación.

Esta vulnerabilidad permitía que un usuario sin privilegios pudiera inyectar y sobrescribir datos en archivos de solo lectura, incluyendo procesos que se ejecutaban como root. A diferencia de Dirty COW, esta vulnerabilidad era considerablemente más sencilla de explotar, lo que aumentaba significativamente su nivel de riesgo.

El impacto de Dirty Pipe se extendió más allá de los servidores Linux, afectando también a dispositivos Android que utilizaban versiones del kernel 5.8 o posteriores.

6.2.3 Spectre y Meltdown (CVE-2017-5753 / CVE-2017-5754)

Año de descubrimiento: 2018
Versiones afectadas: Prácticamente todos los sistemas
Linux que corren sobre procesadores Intel, AMD y ARM fabricados desde 1995.
Nivel de gravedad: Crítico

Descripción: Spectre y Meltdown son dos vulnerabilidades relacionadas con el hardware de los procesadores que afectaron a prácticamente todos los sistemas

operativos modernos, incluyendo Linux. Ambas aprovechan una técnica de optimización del procesador llamada ejecución especulativa.

Meltdown permitía que un proceso sin privilegios accediera a la memoria del kernel, exponiendo información sensible del sistema. Spectre, por su parte, permitía a un proceso malicioso engañar a otros procesos para que accedieran a ubicaciones arbitrarias de su propia memoria, pudiendo así filtrar información confidencial.

La corrección de estas vulnerabilidades requirió no sólo parches del kernel de Linux, sino también actualizaciones de microcódigo por parte de los fabricantes de procesadores, lo que implicó en muchos casos una reducción en el rendimiento del sistema.

6.2.4 CVE-2024-1086 (Escalada de privilegios en Netfilter)

Año de descubrimiento: 2024
Versiones afectadas: Kernel de Linux desde la versión 5.14 hasta la 6.6.
Nivel de gravedad: Crítico

Descripción: Esta vulnerabilidad fue detectada en el subsistema Netfilter del kernel de Linux, que es el responsable del filtrado de paquetes de red y la traducción de direcciones (NAT). Un exploit de prueba de concepto fue publicado en GitHub en marzo de 2024, lo que facilitó su explotación por parte de actores maliciosos.

Al explotar esta vulnerabilidad, un atacante podía eludir los controles de seguridad del sistema, obtener acceso administrativo (root) y moverse lateralmente a través de redes. Esta vulnerabilidad fue utilizada activamente en ataques de ransomware, donde los atacantes cifraban archivos críticos, exfiltraban datos sensibles y establecían acceso persistente al sistema comprometido.

6.2.5 CopyFail (CVE-2026-31431)

Año de descubrimiento: 2026
Versiones afectadas: Kernel de Linux desde la versión 4.14 (lanzada en 2017) hasta la versión 6.18.22, incluyendo todas las versiones del kernel 5.x y 6.x anteriores al parche.

Nivel de gravedad: Crítico
Descripción: CopyFail es la vulnerabilidad más reciente y fue descubierta por el investigador Taeyang Lee de la empresa Theori, con la asistencia de herramientas de análisis basadas en inteligencia artificial. El fallo se encuentra en el componente criptográfico authencesn del kernel de Linux, accesible a través de la interfaz de socket AF_ALG.

A diferencia de vulnerabilidades anteriores, CopyFail no requiere técnicas complejas ni condiciones especiales para ser explotada, lo que la hace especialmente peligrosa. Permite a un atacante con acceso básico al sistema escalar privilegios hasta obtener control total (root) del servidor. Su ventana de exposición de casi nueve años implica un impacto potencial en servidores empresariales, instancias en la nube y dispositivos basados en Linux a nivel global.

Distribuciones afectadas confirmadas incluyen Ubuntu, Amazon Linux, RHEL, SUSE, Debian, Arch, Fedora, Rocky y AlmaLinux con kernels en el rango afectado.

6.3. Conclusión

El historial de vulnerabilidades del kernel de Linux demuestra que ningún sistema operativo, por más robusto que sea, está completamente libre de fallos de seguridad. Sin embargo, lo que diferencia a Linux empresarial de otras alternativas es la velocidad y eficiencia con la que los equipos de desarrollo y las distribuciones con soporte oficial publican y distribuyen los parches correctivos. Esto refuerza la decisión de utilizar SUSE Linux Enterprise Server 16 con soporte Premium, ya que garantiza la recepción oportuna de actualizaciones de seguridad y la asistencia técnica necesaria para mitigar cualquier vulnerabilidad que pudiera afectar la infraestructura del Hospital de Clínicas.

7. Configuración de Usuarios y Grupos

7.1. Definición de Roles y Criterio de Permisos

Para garantizar la seguridad de la infraestructura del servidor del Hospital de Clínicas, se aplica el principio de mínimo privilegio (PoLP), el cual consta que cada usuario y grupo posee estrictamente los permisos necesarios para realizar sus tareas, evitando riesgos de seguridad o modificaciones no autorizadas en el sistema operativo. Se definen los siguientes roles del sistema:

- **Administrador del Sistema:** Posee permisos totales de lectura, escritura y ejecución (rwx) sobre la totalidad del sistema de archivos. Requerido para tareas de mantenimiento global, instalación de paquetes y gestión de la seguridad del servidor.
- **Operador de Centro de Cómputos:** Cuenta con permisos de ejecución sobre los scripts de gestión y lectura en logs del sistema. Permite ejecutar tareas operativas sin comprometer la configuración core del sistema operativo.
- **Usuario de Aplicación / Web Server:** Posee permisos de lectura y ejecución sobre el directorio raíz de la aplicación web, y permisos de escritura únicamente en la carpeta destinada a subida de archivos o temporales. Evita que una vulnerabilidad en la aplicación web permita la modificación del código fuente o ejecución de archivos arbitrarios.

7.2. Creación de Grupos Funcionales

Se estructuran los permisos agrupando a los usuarios según sus áreas de responsabilidad operativa:

- grp_sysadmin**:** Integrado por los administradores principales del servidor.

- grp_operadores**:** Integrado por el personal encargado de la ejecución de scripts de monitoreo, respaldo y gestión de servicios.

- grp_backups**:** Grupo con acceso exclusivo de lectura al directorio de la aplicación y de escritura sobre el directorio de almacenamiento de respaldos.

- grp_redes**:** Grupo destinado a la auditoría y configuración de las reglas de cortafuegos (firewall) y parámetros de red.

7.3. Asignación de Permisos sobre Archivos y Directorios

Se establece la matriz de permisos sobre las rutas críticas del servidor:

Ruta de Sistema	Propietario/Grupo	Permisos	Justificación de Seguridad
/var/www/sigsm	sysadmin:www-data	755 (rwx r-x r-x)	Permite lectura y ejecución al servidor web, impidiendo la escritura no autorizada en la raíz del sitio.
/var/www/html/ uploads	www-data:www-data	770 (rwx rwx ---)	Permite al sistema guardar los documentos PDF subidos por los administrativos.
/var/backups	sysadmin:grp_backups	770 (rwx rwx ---)	Restringe el acceso a las copias de seguridad únicamente al personal o scripts autorizados de respaldo.
/scripts/operador/	sysadmin:grp_operadores	750 (rwx r-x ---)	Permite la ejecución del script del centro de cómputos solo a los miembros del grupo operador.

7.4. Uso de Privilegios Elevados

El acceso directo a la cuenta del usuario “root” se encuentra deshabilitado para conexiones remotas y locales.

Usuarios autorizados: Únicamente los integrantes del grupo “grp_sysadmin” y el usuario asignado para la ejecución automatizada del script de respaldos cuentan con permisos en /etc/sudoers.

Criterio de justificación: La delegación de comandos mediante sudo obliga al uso de contraseñas individuales y genera un registro de auditoría imborrable en /var/log/auth.log. Esto permite identificar exactamente qué usuario ejecutó un

comando privilegiado, a qué hora y con qué parámetros, garantizando la trazabilidad total del sistema.

8. Respaldo a Largo Plazo y Alta Disponibilidad

8.1. Selección de medios de almacenamiento para respaldo a largo plazo

Para definir el medio de respaldo más adecuado se evaluaron las cuatro opciones planteadas por la consigna, contrastándolas con el contexto real del proyecto: un único servidor físico alojado en el piso 6 del DTI del Hospital de Clínicas, un presupuesto de software de costo cero (ya establecido en la factibilidad económica de Ingeniería de Software), y datos administrativos sensibles que, si bien no incluyen historias clínicas, sí requieren cumplir con la Ley N° 18.331 de Protección de Datos Personales.

El disco externo se descartó como medio principal porque, aunque es económico y de fácil implementación, depende de una conexión manual y de que un operador recuerde ejecutar la copia y retirar el disco del sitio; si permanece conectado al mismo servidor o en la misma sala, no protege ante un incendio, inundación o robo que afecte físicamente al piso 6. Las cintas magnéticas (tape backup) también se descartaron, ya que si bien siguen siendo el estándar en grandes centros de datos para archivado de décadas, requieren hardware de lectura/escritura específico (unidades LTO) que el Hospital de Clínicas no tiene disponible actualmente, y su costo de implementación no se justifica para el volumen de datos que maneja este sistema, estimado en unos pocos gigabytes anuales entre documentos PDF, registros de traslados y respuestas de encuestas.

Se optó entonces por una estrategia combinada entre NAS y almacenamiento en la nube, cada uno cumpliendo un rol distinto. El NAS se ubica dentro de la propia infraestructura del DTI y permite una restauración rápida ante fallos comunes, como la corrupción de una base de datos o el borrado accidental de un documento, sin depender de una conexión a internet estable. El almacenamiento en la nube, por su parte, cumple la función de copia externa (off-site), que es la que realmente protege al sistema ante un desastre físico que afecte al edificio del hospital. Este riesgo no es meramente teórico: el propio Hospital de Clínicas debió evacuar de urgencia a 70 pacientes hacia pisos superiores durante una emergencia por lluvias en 2023, lo cual confirma que una copia guardada únicamente dentro del mismo edificio queda expuesta al mismo evento que se busca mitigar. Dado que el proyecto ya se apoya en herramientas de código

abierto y de costo cero, se recomienda usar un proveedor de almacenamiento en la nube con capa gratuita o de bajo costo (por ejemplo, un bucket de almacenamiento de objetos) únicamente para alojar las copias comprimidas y cifradas, nunca como repositorio de trabajo del sistema en producción.

8.2. Estrategia de retención

Se define una política de retención escalonada, del tipo abuelo-padre-hijo, que equilibra la necesidad de poder recuperar información reciente con rapidez y la de conservar un historial más amplio sin que el volumen de copias crezca indefinidamente. Se conservan copias diarias incrementales durante los últimos siete días, lo que permite volver a cualquier punto de la última semana en caso de un error operativo detectado tarde, como una carga incorrecta de documentos o una modificación errónea de un registro de traslado. Sobre esas copias diarias se generan copias semanales completas, que se conservan durante cuatro semanas, cubriendo así todo el mes en curso. A su vez, se conserva una copia mensual completa por cada uno de los últimos doce meses, lo que permite reconstruir el estado del sistema en cualquier punto del último año. Finalmente, se conserva una copia anual de forma indefinida, no tanto por una necesidad técnica sino porque, tal como ya se señaló en el análisis de beneficios estratégicos de Emprendedurismo y Gestión, este historial funciona como respaldo administrativo ante una eventual auditoría de la gestión de traslados por parte del hospital.

Este esquema implica conservar, en todo momento, entre 7 y 10 copias activas de forma simultánea (los diarios de la semana, los semanales del mes y los mensuales del año), lo cual es perfectamente manejable dado el volumen de datos reducido del sistema y el espacio de 1 TB disponible en el servidor.

8.3. Mecanismos de alta disponibilidad

En este punto es importante ser honestos con la escala real del proyecto: se trabaja sobre un único servidor físico, no sobre un centro de datos con múltiples nodos, por lo que no todos los mecanismos de alta disponibilidad aplican de la misma manera.

El mecanismo más viable dentro de ese único servidor es RAID, específicamente RAID 1 (espejado) sobre el almacenamiento SSD. Esto implica que los datos se escriben simultáneamente en dos discos físicos idénticos, de forma que si uno falla, el sistema sigue funcionando de forma transparente con el segundo disco mientras se gestiona el reemplazo, sin que esto represente una interrupción del

servicio ni pérdida de información. Es la forma más económica y realista de dar continuidad ante una falla de hardware dentro de la infraestructura ya definida en la primera entrega, sin necesidad de adquirir un segundo servidor completo.

Por encima del RAID, se contempla la replicación Master-Slave de MySQL/MariaDB, tal como ya se había anticipado como logro de la tercera entrega en la letra del proyecto. En este esquema, el servidor de base de datos principal (Master) replica en tiempo casi real todas las escrituras hacia un segundo servidor (Slave), que se mantiene como una copia activa y actualizada de la base de datos. Si el Master deja de responder, el Slave puede promoverse manualmente (o de forma semiautomática) a servidor principal, reduciendo drásticamente el tiempo de interrupción del sistema en comparación con tener que restaurar desde una copia de respaldo. Este mecanismo es especialmente relevante para el módulo de trazabilidad de ambulancias, dado que una caída de la base de datos en medio de un traslado activo dejaría al personal administrativo sin visibilidad del estado de la flota.

En cuanto al clustering, se lo considera fuera del alcance real del proyecto en esta etapa. El clustering implica coordinar múltiples servidores para que se comporten como un único servicio con balanceo de carga y conmutación automática por error, lo cual requiere una inversión en hardware e infraestructura de red que excede tanto el presupuesto de costo cero definido para el proyecto como la infraestructura física de un único servidor ya acordada con el DTI del Hospital de Clínicas. Se lo deja mencionado como una posible línea de evolución futura del sistema, en caso de que la institución decida escalar la solución más allá del proyecto de egreso.

8.4. Plan de recuperación ante desastres (RTO/RPO)

Para definir los objetivos de recuperación se diferenció entre los dos módulos del sistema, ya que no tienen el mismo nivel de criticidad operativa. El módulo de trazabilidad de ambulancias es el más sensible al tiempo, porque una interrupción prolongada obliga al personal administrativo a volver a la coordinación telefónica manual que el sistema busca reemplazar. Para este módulo se define un RTO (Recovery Time Objective) de 4 horas, es decir, el tiempo máximo aceptable para que el sistema vuelva a estar operativo tras una falla, y un RPO (Recovery Point Objective) de 30 minutos, que representa la cantidad máxima de información que se puede permitir perder, cubierta por la replicación Master-Slave descripta en el punto anterior.

El módulo de gestión documental y encuestas, al ser de naturaleza más informativa y no depender de una actualización constante durante el día, admite objetivos más laxos: un RTO de 24 horas y un RPO de 24 horas, alineado directamente con la frecuencia de las copias diarias incrementales definidas en la estrategia de retención.

Definir estos valores no tiene sentido si no se verifica periódicamente que las copias realmente puedan restaurarse. Por eso se establece una prueba de restauración mensual, en la que se toma una copia semanal completa y se restaura en un entorno de prueba aislado (por ejemplo, una máquina virtual levantada con VirtualBox, tal como ya usan para replicar el entorno de producción), verificando que el sistema arranque correctamente y que los datos restaurados sean consistentes. El resultado de cada prueba se documenta en un acta breve, siguiendo el mismo formato de actas de reunión que ya utiliza el equipo, dejando constancia de la fecha, la copia utilizada, el tiempo real que tomó la restauración y cualquier incidencia encontrada. Esto permite detectar a tiempo si una copia está corrupta o si el procedimiento de restauración necesita ajustarse, en lugar de descubrirlo recién en el momento de una falla real.

9. Mecanismos de Backup para los Datos de la Aplicación

9.1. Descripción general del script

El módulo de Gestión de Respaldos se implementó como un script de Bash interactivo (*Gestión de respaldos.sh*), pensado para integrarse como una opción más dentro del script general de operador del centro de cómputos que exige la letra del proyecto. Su punto de entrada es la función *menu_respaldos*, que presenta un menú con cinco opciones: realizar un respaldo completo, realizar un respaldo incremental, programar respaldos periódicos mediante cron, restaurar datos desde un respaldo existente, y volver al menú principal. Cada una de estas acciones se apoya en dos funciones auxiliares transversales: *log_evento*, que registra con fecha y hora cada operación (éxito o error) en un archivo de log centralizado, y *confirmar_accion*, que exige una confirmación explícita del operador antes de ejecutar cualquier acción potencialmente destructiva, como una restauración que sobrescriba archivos existentes.

9.2. Backup de archivos propios de la aplicación (configuraciones, uploads, logs)

El script no tiene hardcodeada una única carpeta de la aplicación, sino que trabaja sobre una variable de origen configurable (*DIR_ORIGEN_DEF*), que por defecto apunta a */var/www/hospital/documentos* pero que el operador puede sobreescribir en cada ejecución. Esto es lo que permite aplicar el mismo script a las distintas categorías de archivos propios de la aplicación que pide la consigna: basta con ejecutar el respaldo indicando como origen la carpeta *config/* para respaldar las configuraciones, *public/uploads/* para los documentos y códigos QR cargados por los administradores, o *logs/app/* para los logs de auditoría del sistema.

En la práctica, para que un mismo respaldo cubra las tres categorías en una sola ejecución (y para que la automatización por cron, que no permite responder prompts interactivos, funcione sin intervención manual), se recomienda fijar *DIR_ORIGEN_DEF* directamente en la carpeta raíz de la aplicación (por ejemplo */var/www/sigsm*), de forma que *rsync* copie en un mismo paso las subcarpetas *config/, public/uploads/* y *logs/app/* que cuelgan de ella.

9.3. Cómo se almacenan los backups

Al iniciarse, el script crea automáticamente la estructura de carpetas de destino (*$DIR_DESTINO_BASE/completos, /incrementales* y */restauraciones*), separando claramente los backups completos de los incrementales. Cada ejecución genera una subcarpeta nueva con marca de tiempo (*full_YYYY-MM-DD_HH-MM-SS* o *inc_YYYY-MM-DD_HH-MM-SS*), de modo que ningún respaldo sobrescribe al anterior y queda un historial navegable de todas las copias realizadas.

La copia en sí se realiza con *rsync -avh*, donde -a (archive) preserva permisos, fechas y enlaces simbólicos de forma recursiva, y *-v* y *-h* muestran el detalle de la operación en formato legible. En el caso del respaldo completo se agrega el flag *--delete*, que hace que el destino sea un espejo exacto del origen en ese momento (si un archivo fue borrado del origen, también desaparece del backup), garantizando que cada backup completo represente el estado real y actual de los datos, sin arrastrar archivos obsoletos.

Cada corrida, exitosa o fallida, queda registrada con fecha y hora en *gestion_respaldos.log*, lo que aporta la trazabilidad necesaria para poder auditar después qué se respaldó, cuándo y con qué resultado.

9.4. Automatización mediante cron

La opción 3 del menú (*programar_cron*) permite instalar las tareas automáticas directamente en el crontab del usuario root, sin salir del script, usando el patrón estándar (*crontab -l; echo "nueva tarea")* | *crontab -,* que agrega la nueva línea conservando las tareas que ya estuvieran programadas.

La frecuencia ofrecida responde a la criticidad de los datos, siguiendo el mismo criterio ya definido en la estrategia de retención del plan de respaldo a largo plazo: un backup incremental diario a las 2:00 AM, ya que los documentos y los logs de auditoría se modifican todos los días y conviene capturar su estado con esa frecuencia; y un backup completo semanal los domingos a las 3:00 AM, suficiente para las configuraciones, que casi no cambian, y como punto de restauración íntegro semanal. Ambas tareas registran su salida en *cron_backup.log*, separado del log de las ejecuciones manuales, lo que permite distinguir a simple vista si un respaldo se generó de forma automática o manual.

9.5. Restauración de datos

La función *restaurar_datos* primero lista los backups completos e incrementales disponibles, para que el operador pueda identificar visualmente cuál restaurar sin tener que recordar nombres de carpetas de memoria. Luego solicita la ruta exacta del backup elegido y la ruta de destino (por defecto, la misma carpeta de origen definida al inicio del script), y antes de ejecutar cualquier cambio exige la confirmación explícita del operador mediante *confirmar_accion.* Esta validación es la que evita una sobrescritura accidental de datos en producción, y es coherente con las pruebas periódicas de restauración que ya se habían definido como práctica mensual en el plan de recuperación ante desastres.

Bibliografía

DesdeLinux. (s.f.). *Dirty Pipe: Una de las vulnerabilidades más graves en años en Linux*. https://blog.desdelinux.net/dirty-pipe-una-de-las-vulnerabilidades-mas-grave-en-anos-en-linux/

S2 Grupo. (2026). *CopyFail (CVE-2026-31431): Vulnerabilidades críticas en entornos Linux durante 2026*. https://s2grupo.es/vulnerabilidades-criticas-linux-2026/

SocPrime. (s.f.). *Vulnerabilidad CVE-2024-1086: Análisis técnico y mitigación*. https://socprime.com/es/blog/vulnerabilidad-cve-2024-1086/

Stackscale. (s.f.). *Protección contra la vulnerabilidad Dirty COW en sistemas Linux*. https://www.stackscale.com/es/blog/proteccion-vulnerabilidad-dirty-cow-linux/

SUSE. (s.f.). *SUSE Linux Enterprise Server 16: AI-Ready y soporte a largo plazo*. https://www.suse.com/es-es/news/suse-linux-enterprise-server-16-ai-ready-long-term-support/

Anexos

I. Primera versión del script adjunto en la carpeta.
II. Segunda versión del script adjunto en la carpeta.
III. GitHub: https://github.com/OblivionBlossom/SIGSM2026